```
0000 separate (IDL.SEM_PHASE)¶    --|-----------------------------------------------------------------------------------------¶package body PRA_WALK is¶  use PRENAME;¶  use VIS_UTIL
     |-----------------------------------------------------------------------------------------¶package body PRA_WALK is¶  use PRENAME;¶  use VIS_UTIL
     ;¶  use REQ_UTIL;¶  use DEF_UTIL;¶  use EXP_TYPE;¶  use EXPRESO;¶  use SET_UTIL;¶  use NOD_WALK;¶¶  PRAGMA_ERROR : exception;¶¶  procedure WALK_PRAGMA
     _ARGUMENTS (USED_NAME_ID, GEN_ASSOC_S : TREE; H : H_TYPE);¶¶  procedure GET_ARGUMENT_NAME¬(USED_NAME_ID : TREE; ASSOC_LIST : in out SEQ_TYPE; ASSOC_OU
     T : out TREE; NEW_ASSOC_LIST : in out SEQ_TYPE; ARGUMENT_LIST : SEQ_TYPE);¶¶  procedure GET_ARGUMENT_EXP (USED_NAME_ID : TREE; ASSOC_LIST : in out SEQ
     _TYPE; ASSOC_OUT : out TREE);¶¶  procedure MUST_BE_SIMPLE_NAME (EXP : TREE);¶¶  procedure MUST_BE_NAME (EXP¬: TREE);¶¶  procedure WALK_PRAGMA (USED_NA
     ME_ID :¬TREE; GEN_ASSOC_S :¬TREE; H :¬H_TYPE) is¶    DEFLIST     : SEQ_TYPE :=¬LIST (D (LX_SYMREP,¬USED_NAME_ID));¶    DEF¬       : TREE;¶    PRAGMA_DE
     FN : TREE     :=¬TREE_VOID;¶  begin¶¶¬       -- FIND THE PRAGMA_ID¶    while¬not IS_EMPTY (DEFLIST) loop¶       POP¬(DEFLIST,¬DEF);¶       PRAGMA_DEFN :=
      D (XD_SOURCE_NAME,¬DEF);¶       if PRAGMA_DEFN.TY = DN_PRAGMA_ID then¶         exit;¶       else¶         PRAGMA_DEFN¬:= TREE_VOID;¶       end¬if;¶    end
      loop;¶¶¬       -- STORE THE PRAGMA_ID (OR VOID)¶    D (SM_DEFN, USED_NAME_ID,¬PRAGMA_DEFN);¶¶¬       -- IF PRAGMA_ID FOUND¶    if PRAGMA_DEFN /= TREE_V
0010 OID then¶¶¬¬     -- SUPPRESS FATAL ERRORS¶       PRAGMA_CONTEXT := USED_NAME_ID;¶¶¬¬     -- WALK THE ARGUMENTS¶       WALK_PRAGMA_ARGUMENTS (USED_NAME_ID,
      GEN_ASSOC_S, H);¶¶¬¬     -- ENABLE FATAL¬ERRORS¶       PRAGMA_CONTEXT := TREE_VOID;¶¶¬¬     -- IF¬ERROR IN PRAGMA, PUT OUT IGNORED MESSAGE¶       if D (S
     M_DEFN, USED_NAME_ID) = TREE_VOID then¶         raise PRAGMA_ERROR;¶       end¬if;¶¶¬¬     -- ELSE -- SINCE PRAGMA_ID NOT FOUND¶    else¶¶¬¬     -- PUT OU
     T ERROR¶       WARNING (D (LX_SRCPOS, USED_NAME_ID), "PRAGMA NOT KNOWN TO IMPLEMENTATION - " & PRINT_NAME (D¬(LX_SYMREP, USED_NAME_ID)));¶    end if;¶¶
       exception¶¶¬       -- IN CASE OF¬ERROR IN ARGUMENT EVALUATION¶    when PRAGMA_ERROR =>¶¶¬¬     -- CLEAR THE PRAGMA_ID¶       D (SM_DEFN, USED_NAME_ID,
     TREE_VOID);¶¶¬¬     -- ENABLE FATAL¬ERRORS¶       PRAGMA_CONTEXT := TREE_VOID;¶¶       WARNING (D (LX_SRCPOS, USED_NAME_ID), "PRAGMA IGNORED - " & PRINT_
     NAME (D¬(LX_SYMREP, USED_NAME_ID)));¶  end WALK_PRAGMA;¶¶  procedure WALK_PRAGMA_ARGUMENTS (USED_NAME_ID, GEN_ASSOC_S : TREE; H : H_TYPE) is¶    PRAGM
     A_ID¬ : TREE¬   := D (SM_DEFN, USED_NAME_ID);¶    ARGUMENT_ID_LIST : SEQ_TYPE := LIST¬(D (SM_ARGUMENT_ID_S, PRAGMA_ID));¶    ASSOC_LIST¬ : SEQ_TYPE :=
     LIST¬(GEN_ASSOC_S);¶    ASSOC_NODE¬ : TREE;¶    ASSOC_EXP¬ : TREE;¶    ASSOC_TYPE¬ : TREE;¶    DEFSET¬ : DEFSET_TYPE;¶    DEFINTERP¬ : DEFINTERP_TYPE;
     ¶    TYPESET¬ : TYPESET_TYPE;¶    DEF¬¬ : TREE;¶    ID¬¬ : TREE;¶    IDLIST¬ : SEQ_TYPE := (TREE_NIL, TREE_NIL);¶    NEW_ASSOC_LIST¬ : SEQ_TYPE := (TR
0020 EE_NIL, TREE_NIL);¶  begin¶¶    case DEFINED_PRAGMAS'VAL (DI (XD_POS, D (SM_DEFN, USED_NAME_ID)))¬is¶¶      when CONTROLLED =>¶¬¬¬  -- $$$$¬IMMEDIATEL
     Y WITHIN DECLARATIVE PART OR PACKAGE SPECIFICATION¶       GET_ARGUMENT_EXP (USED_NAME_ID,¬ASSOC_LIST, ASSOC_EXP);¶       MUST_BE_SIMPLE_NAME (ASSOC_
     EXP);¶       ASSOC_EXP¬    := WALK_TYPE_MARK (ASSOC_EXP);¶       NEW_ASSOC_LIST := APPEND (NEW_ASSOC_LIST,¬ASSOC_EXP);¶       ASSOC_TYPE¬    := GET_B
     ASE_STRUCT (ASSOC_EXP);¶       if ASSOC_TYPE.TY /= DN_ACCESS or else D (SM_DERIVED, ASSOC_TYPE) /= TREE_VOID then¶¬ERROR (D (LX_SRCPOS, ASSOC_EXP), "
     CONTROLLED NOT ALLOWED FOR TYPE");¶       elsif D (XD_REGION, D¬(XD_SOURCE_NAME, ASSOC_TYPE))¬/= D (XD_SOURCE_NAME, H.REGION_DEF) then¶¬ERROR (D (LX_
     SRCPOS, ASSOC_EXP), "CONTROLLED NOT ALLOWED HERE");¶       else¶¬DB (SM_IS_CONTROLLED, ASSOC_TYPE, True);¶       end if;¶¶      when ELABORATE =>¶¬¬
     ¬  -- $$$$¬ONLY AFTER CONTEXT CLAUSE; MUST BE IN CONTEXT CLAUSE¶       loop¶¬GET_ARGUMENT_EXP (USED_NAME_ID, ASSOC_LIST, ASSOC_EXP);¶¬MUST_BE_SIMPLE_
     NAME¬(ASSOC_EXP);¶¬FIND_DIRECT_VISIBILITY (ASSOC_EXP, DEFSET);¶¬REQUIRE_UNIQUE_DEF (ASSOC_EXP, DEFSET);¶¬ASSOC_EXP¬     := RESOLVE_NAME (ASSOC_EXP, GE
     T_THE_ID (DEFSET));¶¬NEW_ASSOC_LIST := APPEND (NEW_ASSOC_LIST, ASSOC_EXP);¶¬if D (SM_DEFN, ASSOC_EXP).TY not in CLASS_NON_TASK_NAME and¬D (SM_DEFN, AS
     SOC_EXP) /= TREE_VOID then¶¬  ERROR (D (LX_SRCPOS, ASSOC_EXP), "LIBRARY UNIT NAME REQUIRED");¶¬end if;¶¬exit when¬IS_EMPTY (ASSOC_LIST);¶       end l
0030 oop;¶¶      when INLINE =>¶¬¬¬  -- $$$$¬AT PLACE OF DECLARATIVE ITEM OR FOLLOWING LIBRARY¬UNIT¶       loop¶¬GET_ARGUMENT_EXP (USED_NAME_ID, ASSOC_LIS
     T, ASSOC_EXP);¶¬MUST_BE_SIMPLE_NAME¬(ASSOC_EXP);¶¬FIND_DIRECT_VISIBILITY (ASSOC_EXP, DEFSET);¶¬while not¬IS_EMPTY (DEFSET) loop¶¬  POP (DEFSET, DEFINT
     ERP);¶¬  DEF := GET_DEF (DEFINTERP);¶¬  ID  := D (XD_SOURCE_NAME, DEF);¶¬  if D (XD_REGION_DEF, DEF) =¬H.REGION_DEF and then (ID.TY in CLASS_SUBPROG_N
     AME or else (¶ID.TY = DN_GENERIC_ID and then¶D (SM_SPEC, ID).TY in DN_PROCEDURE_SPEC¬.. DN_FUNCTION_SPEC)) then¶¬    IDLIST := APPEND (IDLIST,¬ID);¶¬
        DB (SM_IS_INLINE, ID, True);¶¬  end if;¶¬end loop;¶¬if IS_EMPTY (IDLIST) then¶¬  ERROR (D (LX_SRCPOS, ASSOC_EXP), "NO SUCH SUBPROGRAM");¶¬end if;¶¬
     D (SM_DEFN, ASSOC_EXP, CAST_TREE (IDLIST));¶¬ASSOC_EXP¬      := MAKE_USED_NAME_ID_FROM_OBJECT (ASSOC_EXP);¶¬NEW_ASSOC_LIST := APPEND (NEW_ASSOC_LIST, A
     SSOC_EXP);¶¬exit when¬IS_EMPTY (ASSOC_LIST);¶       end loop;¶¶      when INTERFACE =>¶¬¬¬  -- $$$$¬AT PLACE OF DECLARATIVE ITEM OR FOLLOWING LIBRARY
     ¬UNIT SPEC¶       GET_ARGUMENT_NAME (USED_NAME_ID, ASSOC_LIST, ASSOC_EXP, NEW_ASSOC_LIST,¬ARGUMENT_ID_LIST);¶       GET_ARGUMENT_EXP (USED_NAME_ID,¬
     ASSOC_LIST, ASSOC_EXP);¶       MUST_BE_NAME (ASSOC_EXP);¶       FIND_VISIBILITY (ASSOC_EXP, DEFSET);¶       while not IS_EMPTY (DEFSET) loop¶¬POP (
     DEFSET, DEFINTERP);¶¬DEF := GET_DEF (DEFINTERP);¶¬ID  := D (XD_SOURCE_NAME, DEF);¶¬if D (XD_REGION_DEF, DEF) = H.REGION_DEF and then¬ID.TY in CLASS_SU
0040 BPROG_NAME then¶¬  IDLIST := APPEND (IDLIST, ID);¶¬  if D (XD_BODY, ID) /= TREE_VOID or else D (XD_STUB, ID) /= TREE_VOID then¶¬    ERROR¬(D (LX_SRCPO
     S, ASSOC_EXP), "BODY ALREADY GIVEN");¶¬    raise¬PRAGMA_ERROR;¶¬  end if;¶¬end if;¶       end loop;¶       if IS_EMPTY¬(IDLIST) then¶¬ERROR (D (LX_S
     RCPOS, ASSOC_EXP), "NO SUCH SUBPROGRAM");¶       end if;¶       D¬(SM_DEFN,¬ASSOC_EXP, CAST_TREE (IDLIST));¶       ASSOC_EXP¬    := MAKE_USED_NAME_I
     D_FROM_OBJECT (ASSOC_EXP);¶       NEW_ASSOC_LIST := APPEND (NEW_ASSOC_LIST,¬ASSOC_EXP);¶       while not IS_EMPTY (IDLIST) loop¶¬POP (IDLIST, ID);¶¬
     D (SM_INTERFACE, ID, D (SM_DEFN, HEAD (NEW_ASSOC_LIST)));¶       end loop;¶¶      when LIST =>¶¬¬¬  -- $$$$¬NOT GENERATING LISTING¶       GET_ARGUME
     NT_NAME (USED_NAME_ID, ASSOC_LIST, ASSOC_EXP, NEW_ASSOC_LIST,¬ARGUMENT_ID_LIST);¶¶      when MEMORY_SIZE =>¶       ERROR (D (LX_SRCPOS, USED_NAME_ID)
     , "PRAGMA MEMORY_SIZE NOT SUPPORTED");¶       GET_ARGUMENT_EXP (USED_NAME_ID,¬ASSOC_LIST, ASSOC_EXP);¶¶      when OPTIMIZE¬=>¶       GET_ARGUMENT_NA
     ME (USED_NAME_ID, ASSOC_LIST, ASSOC_EXP, NEW_ASSOC_LIST,¬ARGUMENT_ID_LIST);¶¶      when PACK =>¶¬¬¬  -- $$$$¬POSITIONS¬AS FOR REPRESENTATION CLAUSE;¬B
     EFORE REP ATTR¶       GET_ARGUMENT_EXP (USED_NAME_ID,¬ASSOC_LIST, ASSOC_EXP);¶       MUST_BE_SIMPLE_NAME (ASSOC_EXP);¶       ASSOC_EXP¬    := WALK_T
     YPE_MARK (ASSOC_EXP);¶       NEW_ASSOC_LIST := APPEND (NEW_ASSOC_LIST,¬ASSOC_EXP);¶       ASSOC_TYPE¬    := GET_BASE_STRUCT (ASSOC_EXP);¶       if A

0050 SSOC_TYPE.TY not in DN_ARRAY .. DN_RECORD then¶¬ERROR (D (LX_SRCPOS, ASSOC_EXP), "PACK NOT ALLOWED FOR TYPE");¶       elsif D (XD_REGION, D¬(XD_SOURC
     E_NAME, ASSOC_TYPE))¬/= D (XD_SOURCE_NAME, H.REGION_DEF) then¶¬ERROR (D (LX_SRCPOS, ASSOC_EXP), "PACK NOT ALLOWED HERE");¶       else¶¬DB (SM_IS_PACK
     ED, ASSOC_TYPE,¬True);¶       end if;¶¶      when PAGE =>¶       null;¶¶      when PRIORITY¬=>¶¬¬¬  -- $$$$¬TASK OR MAIN PROGRAM¶       GET_ARGUMEN
     T_EXP (USED_NAME_ID,¬ASSOC_LIST, ASSOC_EXP);¶       EVAL_EXP_TYPES (ASSOC_EXP, TYPESET);¶       REQUIRE_TYPE (PREDEFINED_INTEGER, ASSOC_EXP, TYPESET
     );¶       ASSOC_EXP := RESOLVE_EXP (ASSOC_EXP, TYPESET);¶       if GET_STATIC_VALUE (ASSOC_EXP)¬= TREE_VOID then¶¬ERROR (D (LX_SRCPOS, ASSOC_EXP), "
     PRIORITY MUST BE STATIC");¶       end if;¶       NEW_ASSOC_LIST := APPEND (NEW_ASSOC_LIST,¬ASSOC_EXP);¶¶      when SHARED =>¶¬¬¬  -- $$$$¬SAME DECLA
     RATIVE PART OR PACKAGE SPECIFICATION¶       GET_ARGUMENT_EXP (USED_NAME_ID,¬ASSOC_LIST, ASSOC_EXP);¶       MUST_BE_SIMPLE_NAME (ASSOC_EXP);¶
     ASSOC_EXP¬    := WALK_NAME (DN_VARIABLE_ID, ASSOC_EXP);¶       NEW_ASSOC_LIST := APPEND (NEW_ASSOC_LIST,¬ASSOC_EXP);¶       ASSOC_TYPE¬    := GET_BASE
     _STRUCT (ASSOC_EXP);¶       if D (SM_DEFN, ASSOC_EXP) = TREE_VOID then¶¬null;¶       elsif ASSOC_TYPE.TY in CLASS_SCALAR or ASSOC_TYPE.TY = DN_ACCES
     S then¶¬if D (SM_RENAMES_OBJ, D (SM_DEFN, ASSOC_EXP)) = TREE_VOID then¶¬  DB (SM_IS_SHARED,¬D (SM_DEFN, ASSOC_EXP), True);¶¬else¶¬  ERROR (D (LX_SRCPO
0060 S, ASSOC_EXP), "MAY NOT BE SHARED");¶¬end if;¶       elsif ASSOC_TYPE /= TREE_VOID then¶¬ERROR (D (LX_SRCPOS, ASSOC_EXP), "MUST BE SCALAR OR ACCESS T
     YPE");¶       end if;¶¶      when STORAGE_UNIT =>¶       ERROR (D (LX_SRCPOS, USED_NAME_ID), "PRAGMA STORAGE_UNIT NOT SUPPORTED");¶       GET_ARGUM
     ENT_EXP (USED_NAME_ID,¬ASSOC_LIST, ASSOC_EXP);¶¶      when SUPPRESS¬=>¶¬¬¬  -- $$$$¬IMMEDIATELY WITHIN DECL PART OR PACKAGE¬SPEC¶       GET_ARGUMENT_
     NAME (USED_NAME_ID, ASSOC_LIST, ASSOC_EXP, NEW_ASSOC_LIST,¬TAIL (ARGUMENT_ID_LIST));¶       if not IS_EMPTY (ASSOC_LIST) then¶¬POP (ASSOC_LIST, ASSOC
     _NODE);¶¬if ASSOC_NODE.TY = DN_ASSOC then¶¬  if D (LX_SYMREP, D (AS_USED_NAME, ASSOC_NODE)) /= D (LX_SYMREP, HEAD (ARGUMENT_ID_LIST)) then¶¬    ERROR¬
     (D (LX_SRCPOS, ASSOC_NODE), "SELECTOR MUST BE ON =>");¶¬    raise¬PRAGMA_ERROR;¶¬  end if;¶¬  D (SM_DEFN, D (AS_USED_NAME, ASSOC_NODE), HEAD (ARGUMENT
     _ID_LIST));¶¬  ASSOC_EXP := D (AS_EXP, ASSOC_NODE);¶¬else¶¬  ASSOC_EXP := ASSOC_NODE;¶¬end if;¶¬if ASSOC_EXP.TY = DN_STRING_LITERAL then¶¬  ASSOC_EXP
     := MAKE_USED_OP_FROM_STRING¬(ASSOC_EXP);¶¬end if;¶¬MUST_BE_NAME (ASSOC_EXP);¶¬FIND_VISIBILITY (ASSOC_EXP, DEFSET);¶¬REQUIRE_UNIQUE_DEF (ASSOC_EXP, DEF
     SET);¶¬ID¬:= GET_THE_ID (DEFSET);¶¬ASSOC_EXP¬:= RESOLVE_NAME (ASSOC_EXP, ID);¶¬if ID.TY in CLASS_OBJECT_NAME'FIRST .. DN_GENERIC_ID and then ID.TY /=
     DN_PACKAGE_ID then¶¬  null;¶¬elsif ID /= TREE_VOID then¶¬  ERROR (D (LX_SRCPOS, ASSOC_EXP), "SUPPRESS NOT ALLOWED ON THIS");¶¬end if;¶¬if ASSOC_NODE.T
0070 Y = DN_ASSOC then¶¬  D (AS_EXP, ASSOC_NODE, ASSOC_EXP);¶¬else¶¬  ASSOC_NODE := ASSOC_EXP;¶¬end if;¶¬NEW_ASSOC_LIST := APPEND (NEW_ASSOC_LIST, ASSOC_NO
     DE);¶       end if;¶¶      when SYSTEM_NAME =>¶       ERROR (D (LX_SRCPOS, USED_NAME_ID), "PRAGMA SYSTEM_NAME NOT SUPPORTED");¶       GET_ARGUMENT_
     EXP (USED_NAME_ID,¬ASSOC_LIST, ASSOC_EXP);¶¶      when PRENAME.DEBUG =>¶       GET_ARGUMENT_NAME (USED_NAME_ID, ASSOC_LIST, ASSOC_EXP, NEW_ASSOC_LIST
     ,¬ARGUMENT_ID_LIST);¶       case LIST_ARGUMENTS'VAL (DI (XD_POS, D (SM_DEFN, ASSOC_EXP)))¬is¶¬when OFF =>¶¬  IDL.DEBUG := False;¶¬when ON =>¶¬  IDL.D
     EBUG := True;¶       end case;¶¶    end case;¶    if not IS_EMPTY¬(ASSOC_LIST) then¶      WARNING (D (LX_SRCPOS, USED_NAME_ID), "TOO MANY PRAGMA ARGU
     MENTS");¶      raise PRAGMA_ERROR;¶    end if;¶¶    LIST (GEN_ASSOC_S, NEW_ASSOC_LIST);¶¶  end WALK_PRAGMA_ARGUMENTS;¶¶  procedure GET_ARGUMENT_NAME¬(
     USED_NAME_ID : TREE; ASSOC_LIST : in out SEQ_TYPE; ASSOC_OUT : out TREE; NEW_ASSOC_LIST : in out SEQ_TYPE; ARGUMENT_LIST : SEQ_TYPE) is¶    TEMP_ARGUM
     ENT_LIST : SEQ_TYPE := ARGUMENT_LIST;¶    ARGUMENT_ID¬   : TREE;¶    ACTUAL_SYM¬    : TREE;¶    ASSOC_EXP¬    : TREE;¶  begin¶    GET_ARGUMENT_EXP (USED
     _NAME_ID, ASSOC_LIST, ASSOC_EXP);¶¬--  IF ASSOC_EXP.TY =¬DN_USED_OBJECT_ID THEN¶--¬--  PUT_LINE ( "USED NAME ID : " & PRINT_NAME ( D(LX_SYMREP, USED_
     NAME_ID) ) );¶---¬  PUT_LINE ( "ASSOC EXP : " &¬PRINT_NAME ( D(LX_SYMREP,ASSOC_EXP) ) );¶--¶--¶---¬     WARNING(D(LX_SRCPOS,USED_NAME_ID),¶--¬---¬     "AR
0080 GUMENT ID REQUIRED");¶---¬     RAISE PRAGMA_ERROR;¶---¬  END IF;¶¶    ACTUAL_SYM  := D (LX_SYMREP, ASSOC_EXP);¶    ARGUMENT_ID := TREE_VOID;¶    while¬n
     ot IS_EMPTY (TEMP_ARGUMENT_LIST) loop¶      if D (LX_SYMREP, HEAD (TEMP_ARGUMENT_LIST))¬= ACTUAL_SYM then¶        ARGUMENT_ID¬:= HEAD (TEMP_ARGUMENT_L
     IST);¶        exit;¶      end¬if;¶      TEMP_ARGUMENT_LIST := TAIL (TEMP_ARGUMENT_LIST);¶    end loop;¶    if ARGUMENT_ID = TREE_VOID then¶      WARNI
     NG (D (LX_SRCPOS, USED_NAME_ID), "ARGUMENT ID INVALID");¶      raise PRAGMA_ERROR;¶    end if;¶¶    D (SM_DEFN, ASSOC_EXP, ARGUMENT_ID);¶    NEW_ASSOC
     _LIST := APPEND (NEW_ASSOC_LIST, MAKE_USED_NAME_ID_FROM_OBJECT (ASSOC_EXP));¶¶    ASSOC_OUT := ASSOC_EXP;¶  end GET_ARGUMENT_NAME;¶¶  procedure GET_AR
     GUMENT_EXP (USED_NAME_ID : TREE; ASSOC_LIST : in out SEQ_TYPE; ASSOC_OUT : out TREE) is¶    ASSOC_EXP : TREE;¶  begin¶    if IS_EMPTY (ASSOC_LIST) the
     n¶      WARNING (D (LX_SRCPOS, USED_NAME_ID), "ARGUMENT REQUIRED");¶      raise PRAGMA_ERROR;¶    end if;¶¶    POP (ASSOC_LIST, ASSOC_EXP);¶    if ASS
     OC_EXP.TY¬= DN_STRING_LITERAL¬then¶      ASSOC_EXP := MAKE_USED_OP_FROM_STRING (ASSOC_EXP);¶    end if;¶¶    ASSOC_OUT := ASSOC_EXP;¶  end GET_ARGUMEN
     T_EXP;¶¶  procedure MUST_BE_SIMPLE_NAME (EXP : TREE) is¶  begin¶    if EXP.TY /= DN_USED_OBJECT_ID then¶      WARNING (D (LX_SRCPOS, EXP), "SIMPLE NAM
     E REQUIRED");¶      raise PRAGMA_ERROR;¶    end if;¶  end MUST_BE_SIMPLE_NAME;¶¶  procedure MUST_BE_NAME (EXP¬: TREE) is¶  begin¶    if EXP.TY not in
0090 CLASS_DESIGNATOR and then EXP.TY /= DN_SELECTED then¶      WARNING (D (LX_SRCPOS, EXP), "NAME REQUIRED");¶      raise PRAGMA_ERROR;¶    end if;¶  end
     MUST_BE_NAME;¶¶    --|-----------------------------------------------------------------------------------------¶end PRA_WALK;¶
```